Open in app ↗ Sign up Sign in

Medium Search

Affirm Tech Blog · Follow publication

Artificial Intelligence + Machine Learning + Fintech + Risk Management +

Software Engineering +

Learning Richer Credit Representations with Transformers at Affirm

11 min read · Sep 17, 2026

Affirm Technology Follow

Listen Share

Author: Kelli-Jean Chun, Machine Learning and Decision Platform, Affirm



Introduction

Every checkout on Affirm presents a different underwriting question. For consumers who haven't yet established a repayment history with us, their credit reports are an especially valuable source of information. Those reports may contain years of borrowing and repayment history with other lenders. Our models combine

that information with previous Affirm history (if a user is an existing Affirm customer) and merchant context to assess the credit risk for a particular transaction.

For years, we captured these signals through hand-engineered features and tree-based models such as XGBoost. This approach delivered substantial improvements across successive generations of underwriting models. But as those models matured, further gains increasingly depended on finding and developing new features. Each addition required domain expertise and substantial time backfilling new features for training new models.

Credit reports contain considerably more detail than any fixed collection of summary features can capture. Individual accounts have their own characteristics and payment histories, and the relationships across accounts can also be informative. Making more of that information available to a model creates an opportunity to learn useful patterns without specifying each one in advance.

We developed a transformer-based architecture to take advantage of that opportunity. The transformer learns representations from detailed credit report information, which we combine with established underwriting features in an XGBoost model.

In offline evaluation, **this transformer-based architecture delivered 1.8x the gain in $-weighted FPDQ60 AUC achieved by our next XGBoost candidate**, with both gains measured against the same production baseline. Among thin file users, users with limited credit history and no FICO score, the transformer-based model delivered 2.1x the gain.

FPDQxx denotes first-payment delinquency: the first scheduled payment becoming at least xx days past due. For example, FPDQ60 uses a 60-day threshold, while FPDQ4 uses a four-day threshold.

In an online A/B test, **this model increased conversion by +1.2 percentage points (3.4% relative) by incrementally approving eligible applications the existing model would have declined**. The additional loans performed better than a comparable expansion under our previous XGBoost models. The new model is online underwriting new users, including both approvals and declines. This expands the model's role beyond incremental approvals through a second look use case to a first look use case, and will soon be rolled out for returning users.

In this post, we explain the architecture, the engineering required to bring it into production, and what we learned from evaluating it.

Solution

We initially focused on consumers who haven't yet established a repayment history with Affirm. For this model, that means they have not yet completed a scheduled repayment with us. We refer to this group as new users throughout the post. With limited internal repayment history to draw on, our goal was to learn more from the account records and payment histories in their credit reports.

Our architecture consists of two sequential stages:

1. **A transformer learns embeddings** (compact numerical representations) primarily from detailed credit report information.
2. **An XGBoost model combines these embeddings with established underwriting features** to produce the risk score used by our decisioning system.

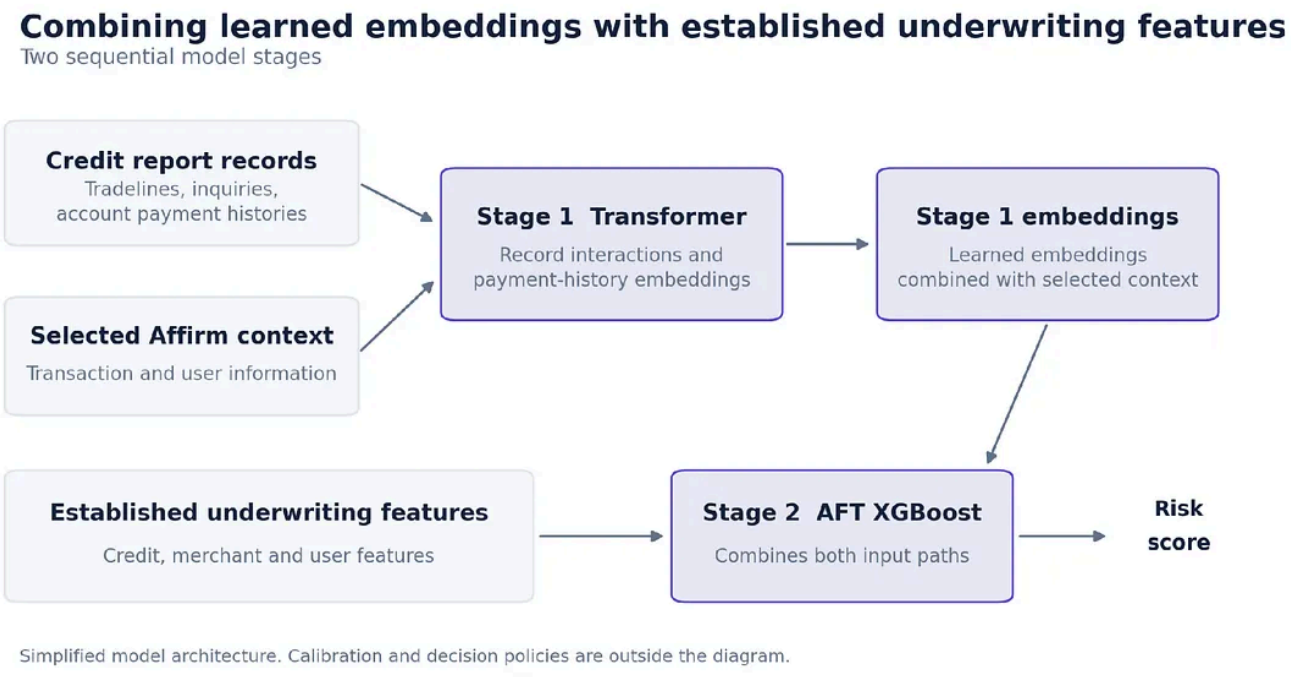


Figure 1. The hybrid architecture combines the Stage 1 Transformer representation with established underwriting features in an AFT XGBoost model.

Representing credit reports as model inputs

A credit report contains several types of structured information. Tradelines describe individual credit accounts, including their characteristics and payment histories. Inquiries record requests for credit information.

We represent each of these records as tokens that the transformer can process. Here, a token represents a structured record rather than a word. For a tradeline, that representation combines numerical attributes, categorical attributes, and an encoding of its payment history.

The payment history introduces an additional level of structure. Monthly payment statuses form a sequence within an account. We encode that sequence with information about each status and its position in the history, then incorporate the resulting representation into the account's token.

This gives the model access to patterns within an account as well as relationships across records and across time, including the dynamic temporal changes of a user's credit profile. Consider the illustrative credit-report histories in Figure 2. Neither consumer has completed a scheduled repayment with Affirm, but both have two credit accounts reported by the bureaus and two late account-months over a six-month period. Consumer A's late payments occurred earlier, on one account. Consumer B's occurred more recently, across two accounts. The total count is identical, but the underlying histories differ.

A feature explicitly designed to capture recency or account-level patterns could preserve some of those distinctions. Our transformer-based model gives us a way to learn additional patterns from the records without designing a separate summary for every pattern, including credit-relevant patterns that we have not previously identified.



Figure 2. A toy example of credit report payment histories. Matching summary statistics such as number of late payments do not imply identical underlying histories.

Learning embeddings for underwriting

We train the transformer using repayment outcomes, so its embeddings are learned for the underwriting task. Attention allows information from different records to interact, and the model produces a compact representation that summarizes useful information from its inputs. Furthermore, our temporal modeling within the transformer block allows for temporal changes of the credit report to be encoded and used within the model.

After training the transformer, we extract the embeddings before the prediction heads and use it as an input to the second-stage model.

The second stage, an accelerated failure time (AFT) XGBoost model form, combines the transformer-derived embeddings with established credit, merchant, and user features.

This design allows us to add learned representations while continuing to use the features and modeling framework developed across previous underwriting releases. In our model development, the hybrid approach outperformed the transformer-only model. In particular, the transformer-only model never reached a performance level above the best XGBoost model that would have been worth the effort to productionize, while the hybrid model provided the 1.8x gain in $-weighted FPDQ60 performance.

What we learned about training credit embeddings

We explored transformer encoders and encoder–decoder architectures, with training approaches including self-supervised masked feature modeling (MFM) and supervised transformers on repayment labels, such as a BERT-style transformer encoder using multi-task supervised learning. The supervised models used multiple prediction heads to learn from both early repayment outcomes, such as FPDQ4, and longer-term outcomes, such as reaching DQ60 within a defined repayment window.

DQxx denotes a loan reaching at least xx days past due within the specified repayment window. Unlike FPDQxx, it is not limited to the first scheduled payment.

Our initial comparisons used the same input set and time periods, and the XGBoost trained on top of the embeddings was held fixed. We evaluated the resulting

embeddings through their contribution to the downstream hybrid model.

Transformer training	FPDQ60 AUC Gain (bps)
Self-Supervised (baseline)	–
Self-Supervised, optimized*	+25
Supervised multi-head training	+124
Supervised multi-head training, optimized*	+187

* Note: less research time was spent optimizing the self-supervised method given the outsized lift supervised approach already showed.

Challenges

Bringing the model into production required substantial work across data, training, serving, latency, and explainability. The system needed to produce a reliable risk score within checkout latency requirements and support model outcomes that are explainable.

Data: preserving detail and consistency

Working with individual credit report records makes data preparation a central part of the modeling system. Reports contain different numbers of accounts and inquiries, categorical fields need consistent encodings, and missing information needs an explicit representation.

Get Affirm Technology's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Our processing converts these records into model inputs, normalizes numerical fields, and masks padded record positions. Historical examples also need to reflect

the information available when the underwriting decision was made.

The same interpretation of those inputs must carry through to online inference. A difference in categorical encoding, missing-value handling, or temporal calculations can change the model's output even when the underlying credit information appears identical.

Offline training infrastructure

Training on detailed records and payment histories introduced a different computational workload from our tree-based models. We needed to make both data loading and neural-network training efficient enough to support research iteration.

The training pipeline supports distributed GPU training, mixed precision, and streaming data from storage. Data-loading workers also hold decoded records and prefetched batches in host memory. Limiting prefetch and loading only required columns help control that footprint. Mixed precision reduces the GPU memory used during neural-network training, while gradient accumulation supports a larger effective batch without fitting that entire batch on a GPU simultaneously. These choices let us scale training while managing the separate memory demands of data preparation and the model.

Online serving infrastructure

The production artifact needed to reproduce the behavior evaluated during development. Exporting a trained model was therefore accompanied by validation of its outputs.

The broader serving integration also needed to connect data processing, transformer inference, and the downstream XGBoost model within the checkout decision path.

We selected the production model based on the best balance of predictive performance, online data availability, latency, and reliability.

Latency

Meeting checkout latency service-level objectives was a requirement for launch. The model's additional computation had to fit within a sub-second latency budget for real-time underwriting decisions.

The relevant latency includes the full computation path from preparing model inputs through generating the final score. Model complexity, input processing, and serving infrastructure therefore had to be considered together to meet those requirements. In order to meet the latency requirements, not only did the model development take this into account (e.g. by keeping the number of transformer model parameters small), we made infrastructure improvements in the realtime serving path that included optimizations on both CPUs and GPUs for model serving.

Explainability

The hybrid architecture of stacking a transformer with a XGBoost introduced a new explainability challenge. Underwriting models need to explain the outcome for each user and provide the principal reason(s) that led to an application being declined. This is a critical piece to be able to use this model for full underwriting decisioning that includes approvals and declines.

We considered several attribution methods, and developed a proprietary algorithm for the hybrid model to be able to explain the model's decisions with the same rigor as our existing approach for XGBoost models. Validation and ongoing monitoring help ensure the model's reasons are accurate and reliable.

Results

We evaluated the model in two complementary ways: its ability to rank credit risk on held-out data, and the effect of using its scores in production decisions. The offline results below come from model development; the online results come from A/B testing.

A larger gain than the next XGBoost iteration

How much progress did the hybrid represent in an already highly optimized underwriting system? We compared the improvements from two candidates: the next XGBoost research iteration and the hybrid transformer + XGBoost. Both were evaluated against the same production XGBoost baseline, on the same held-out loans. The next iteration provides a concrete example of a conventional model release; it does not establish an average across releases.

Our new transformer model was substantially better than the old XGBoost-based model in differentiating delinquent (high-risk) from non-delinquent loans (low-risk), as measured through first payment DQ60 ROC-AUC in a 1 month time-holdout segment of captured loans. Additionally, we evaluated on longer term metrics and observed a similar pattern of improved risk differentiation for our new model.

The hybrid model delivered **about 1.8x the improvement** of the next XGBoost iteration. Both candidates were trained over the same period. Relative to production AUC, those overall improvements are **2.16% for the next XGBoost iteration and 3.83% for the hybrid**. Figure 3 shows the relative gains across delinquency horizons. These percentages measure improvements in risk ranking.

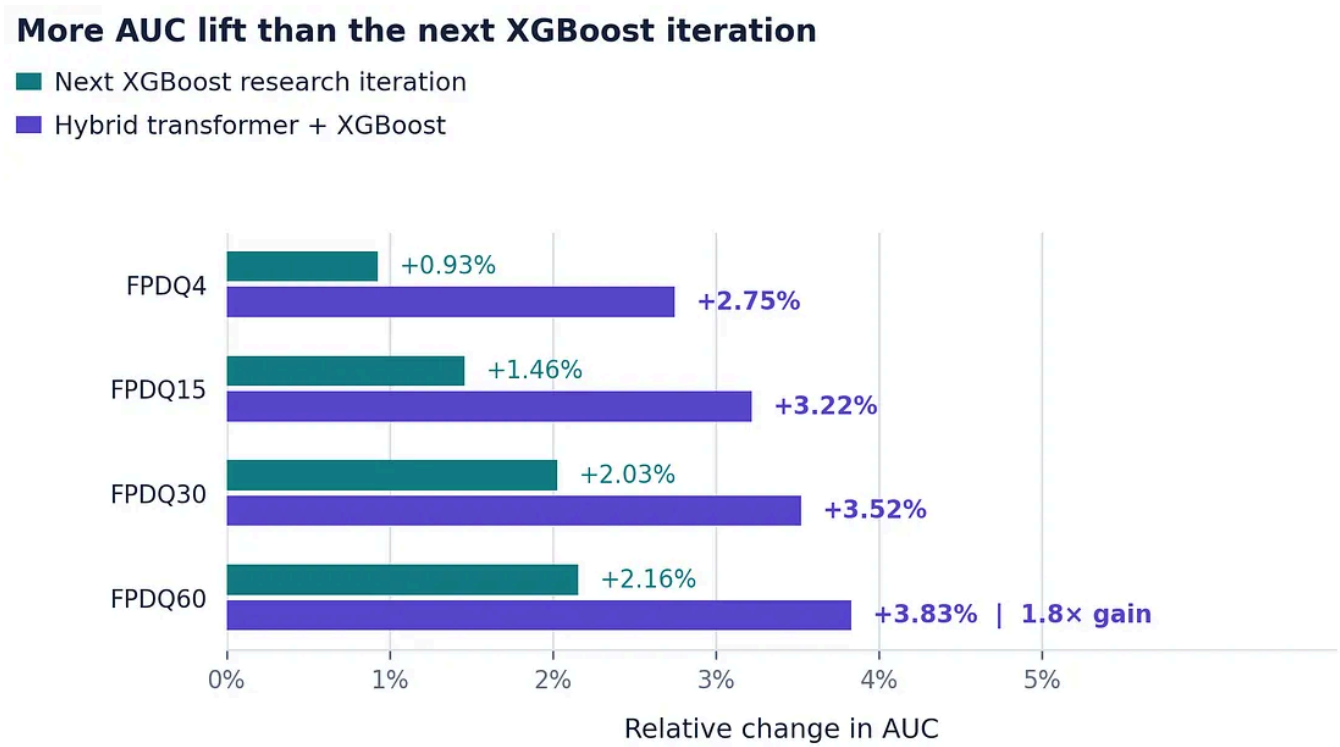


Figure 3. Overall relative AUC improvements over production.

A modelling lesson from thin file users

The largest gain across the credit-score segments we evaluated appeared among thin-file consumers. Here, "thin file" means credit-report information is present, but the history is insufficient to generate a FICO score. The hybrid delivered **about 2.1 times the FPDQ60 AUC gain of the next XGBoost iteration** in this segment.

This finding sharpened the motivation for learning from detailed records. A missing summary score does not mean the underlying report is uninformative. Individual accounts and payment histories, even if there may only be a limited set of information available, can still provide useful signal.

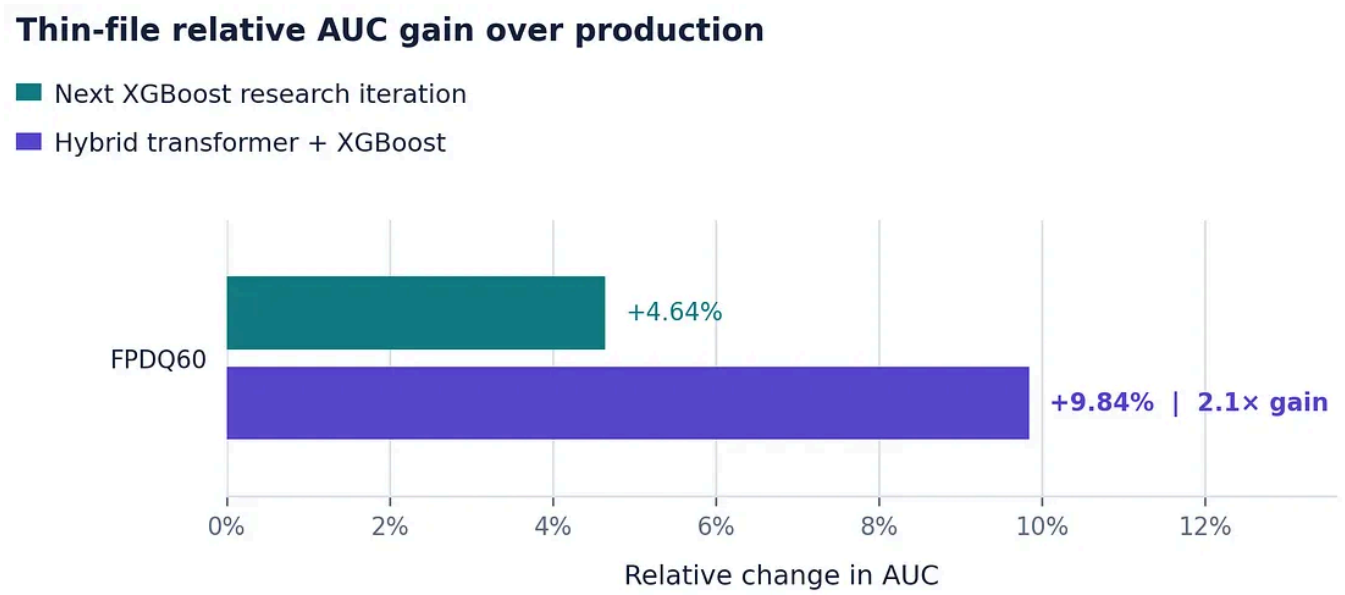


Figure 4. Thin-file relative FPDQ60 AUC improvements over production

Online conversion and the credit tradeoff

We first introduced the transformer-based model through two second-look use cases: (1) approving additional eligible applications that the existing decisioning system would have declined as mentioned in the introduction, and (2) optimizing down-payments. Each was evaluated against existing decisioning through a controlled experiment. We focus on (1) in this post, but the takeaway from both experiments was the same: the transformer-based model improves the risk-ranking of users over XGBoost.

Across eligible new users, the incremental-approval experiment increased end-to-end conversion by **+1.2 percentage points (3.4% relative)**. The incrementally approved loans performed better than a comparable expansion under our previous machine learning models.

The second-look experiment allowed the model to incrementally approve, but did not let the transformer-based model decline applications approved by the existing system. It therefore measured a limited use of the model's additional signal. We are now running a first look experiment using the model for full underwriting decisions for new users, including approvals and declines. Given the outsized offline and online performance already observed, we are also launching the next iteration of the transformer-based model to all users, repeat and new.

Looking ahead

The initial use case for new users established a path for learning richer representations from credit reports within our underwriting architecture. From

here, we see opportunities to expand the approach across additional decisioning use cases.

A natural next step is a shared credit representation that can support multiple credit tasks, which includes both credit report information and internal Affirm user history. This would allow us to investigate how much of the information learned for one underwriting problem transfers to another, and how shared representations can reduce repeated model and feature development.

Our longer-term research is to learn a shared representation from both bureau credit reports and Affirm's own data, such as loan and repayment events, to improve tasks for multiple domains across Affirm ML. These sources provide complementary views of a consumer's financial behavior. Learning from them together could support a broader range of decisions and reduce the need to rebuild features for each task.

This raises concrete research questions: what transfers across tasks, how representations should adapt as a consumer establishes repayment history with us, and which additional data sources improve decisions. The model described here provides a production starting point for that work.

These directions build on the same opportunity: making more of the information in our data available to models while continuing to improve the systems that turn their predictions into reliable decisions.

Artificial Intelligence + Machine Learning + Fintech +
Risk Management + Software Engineering +

Follow

Written by Affirm Technology

182 followers · 2 following

Follow

Published in Affirm Tech Blog

905 followers · Last published Oct 21, 2026

www.affirm.com/careers